Наследование

Наследование – механизм, позволяющий дочернему классу получить доступ к атрибутам и методам родительского класса, пользоваться и изменять их.[1]

Наследование – Класс может наследовать от другого класса и использовать преимущества атрибутов и методов, определяемых суперклассом.[2]

Проблематика

инстанцирование (instancing)

Наследование в Python

В наследовании нет ничего сокрального. И по большому счету это еще один метод доступа к полям класса.

В python наследование организуется через добавление класса родителя в `__mro__` и `__bases__`
Если кратко, то `__bases__` хранит в себе классы своих родителей, а `__mro__` линейный список всех своих предков.
Подробнее про MRO можете прочитать на соответствующей странице

К примеру:

```python
class A: ...
class B: ...
class C(A, B): ...

C.__bases__
C.__mro__
```

```
(<class '__main__.A'>, <class '__main__.B'>)
```

```
(<class '__main__.C'>, <class '__main__.A'>, <class '__main__.B'>, <class 'object'>)
```

А раз наследование не прибито гвоздями к полу, то сможем ли мы *обмануть* цепочку наследования и добавить к ней не класс, а объект?

Так в python существует метод, благодаря которому можно это реализовать благодаря методу `__mro_entries__`. Подробнее прочитать о нем можно В PEP 560

`__mro_entries__` принимает `tuple` всех базовых классов наследника из всех базовых классов наследника и возвращает `tuple` из классов (то есть экземпляров `type`)

К примеру отнаследуемся от функции:

```python
def my_func():
    print('called')
my_func.__mro_entries__ = lambda bases: (str,)

class MyStr(my_func): ...
```

[1] определение из лекции по ООП от 10.10.26
[2] Объектно-ориентированное мышление – Мэтт Вайсфельд

```
MyStr.__mro__
```

```
(<class '__main__.MyStr'>, <class 'str'>, <class 'object'>)
```

```python
MyStr('abc')
```

```
'abc'
```

То есть тут когда мы наследуемся от функции на самом деле мы наследуемся от другого класса, который скрыт под реализацией.
Такая возможность имеет применения в метапрограммировании

На самом деле такая динамическая структура имеет применение в метапрограммировании, а так же в аннотации обобщенных типов (Generic) в наследовании классов

Реальное применение:

```python
class GenericAlias:
    """
    Фабрика, которая создается при вызове Stack[int].
    Именно она подменяет себя на реальный класс в MRO.
    """
    def __init__(self, origin_class, type_params):
        self.origin_class = origin_class
        self.type_params = type_params

    def __mro_entries__(self, bases):
        # Когда Python видит класс, наследуемый от GenericAlias,
        # он заменяет эту фабрику на оригинальный класс (origin_class).
        return (self.origin_class,)


class Stack:
    """
    Основной класс, поддерживающий синтаксис дженериков.
    """
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def __class_getitem__(cls, params):
        # Метод перехватывает обращение через квадратные скобки (например,
        Stack[int])
```

```
        # и возвращает специальный объект с поддержкой __mro_entries__
        # К слову так же добавлен в PEP 560
        return GenericAlias(cls, params)
```

Создаем новый класс, наследуясь от параметризованного дженерика

```python
class IntStack(Stack[int]):
    def sum_all(self):
        return sum(self.items)

my_stack = IntStack()
my_stack.push(10)
my_stack.push(20)
```

Сумма элементов:

```python
my_stack.sum_all()
```

```
30
```

MRO класса `IntStack`:

```python
IntStack.__mro__
```

```
(<class '__main__.IntStack'>, <class '__main__.Stack'>, <class 'object'>)
```